Strife: A Secure Distributed Payment Gateway

Two-phase commit over gRPC with durable logs, crash recovery and mutual TLS

October 2026

Abstract

Strife moves money between accounts held at independent bank servers. A payment gateway authenticates users and coordinates each transfer with the two-phase commit (2PC) protocol, so a payment either happens at both banks or at neither. Every component talks over mutually authenticated TLS, users hold revocable JWT sessions, and every 2PC vote and decision is written to a SQLite log before it is acted on, so the system recovers from a crash of the gateway or of any bank at any point. A chaos experiment that hard-killed components 15 times during 2,747 payments ended with every payment atomic and the total money unchanged. The coordinator can also run as several gateways sharing a Raft-replicated log in etcd, which removes the blocking weakness of 2PC; 55 automated tests cover payments, security, recovery, storage and the replicated mode. This report describes the design, argues its correctness, and presents the measurements.

1 Project Overview

Strife simulates the core of a payment processor such as Stripe: clients ask a gateway to transfer money, and the gateway debits one bank and credits another. Because the two banks are separate processes with separate storage, the transfer is a distributed transaction. The system meets the following requirements:

- Bank servers, clients and a payment gateway implemented with gRPC.
- Mutual TLS on every connection, with identities checked on both sides.
- Authentication, authorization and logging implemented with gRPC interceptors.
- Idempotent payments: retrying a payment never charges twice.
- Offline payments: queued by the client and sent once the gateway is reachable again.
- Atomic transfers via 2PC with timeouts.
- Fault tolerance: durable state at every participant and automatic recovery after crashes.

2 System Architecture

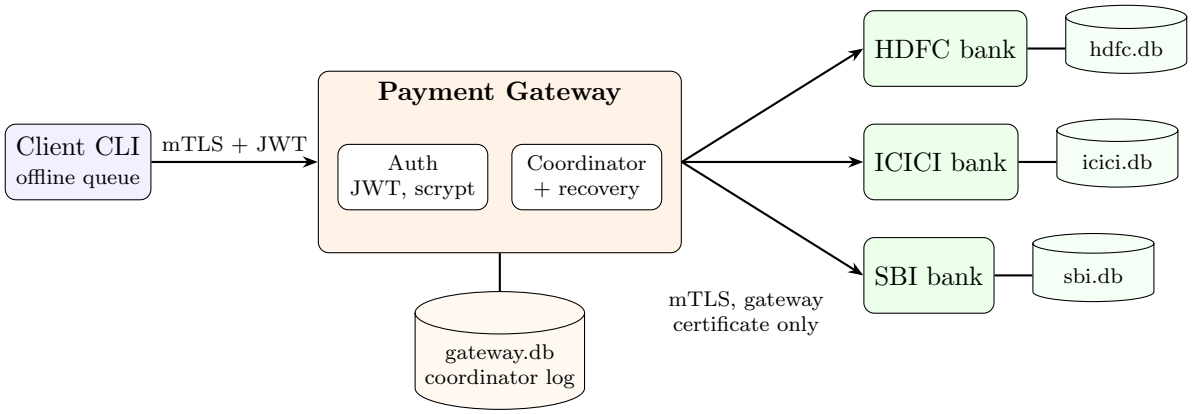


Figure 1: Components and their durable state. Banks also call `RegisterBank` on the gateway at start-up.

2.1 Payment Gateway (`server/payment_gateway.py`)

- Central coordinator: validates requests, runs 2PC and answers queries.
- Persists payments, the bank directory and revoked tokens in `gateway.db` (SQLite, WAL mode, `fsync` on every commit, shared through group commit).
- Runs a *recovery worker* that drives in-doubt payments to a final state (Section 4.5).
- gRPC methods: `AuthenticateClient`, `LogoutClient`, `CheckBalance`, `ProcessPayment`, `GetPaymentStatus`, `ViewTransactionHistory` and `RegisterBank`.

2.2 Bank Servers (`server/bank_server.py`)

- Each bank is an independent process with its own SQLite ledger (`<bank>.db`) holding account balances and the 2PC state of every transaction it has seen.
- Opening balances are loaded from `bank_data.json` the first time a bank starts; afterwards the ledger is the source of truth.
- gRPC methods: `GetBalance`, `PrepareTransaction`, `CommitTransaction` and `AbortTransaction`. Only the gateway's certificate may call them.
- On start-up the bank registers its host and port with the gateway, retrying until the gateway is reachable.

2.3 Client (`client/client.py`)

- Interactive command-line interface for login, payments, balances, history and logout.
- Parses rupee amounts exactly (as decimals) and sends integer paise.
- Queues payments on disk while the gateway is unreachable and sends them automatically when it returns (Section 7).

2.4 Shared modules

`auth.py` (password hashing, login throttling, JWT sessions), `interceptor.py` (authorization, certificate identity, access logging), `tls.py` (mTLS credentials), `storage.py` (serialised SQLite transactions) and `config.py` (settings from environment variables).

2.5 Money representation

All amounts are 64-bit integers in *paise*. Floating point is never used for money, so balances cannot drift through rounding (for example $0.1 + 0.2 \neq 0.3$ in binary floating point).

3 Authentication & Security

3.1 Mutual TLS and component identity

A local certificate authority, created by `scripts/gen_certs.py`, issues one certificate per component. Keys are generated locally (or inside Docker and the test suite) and are never committed to the repository. Both sides of every connection verify the other's certificate (`require_client_auth=True`), and the certificate's common name (CN) is then used for authorization:

Component	Certificate CN	May call
Gateway	`gateway`	bank RPCs (the only identity banks accept)
Bank	`<bank>.bank`, e.g. `hdfc.bank`	`RegisterBank` for *its own* name only
Client	`client`	user RPCs (plus a valid JWT)

Because a bank may only register under the name in its own certificate, a client or a different bank cannot redirect HDFC's traffic to an address it controls. Because banks accept only the gateway's certificate, nobody can call `PrepareTransaction` or `CommitTransaction` directly to create or move money. Conversely, user RPCs accept only the client certificate, so a stolen JWT cannot be used over a bank's or the gateway's certificate.

In the Docker deployment each container mounts a volume holding only its own key, its certificate and `ca.crt`. The CA's private key stays in a volume that no running service mounts, so compromising one bank container exposes neither the gateway's key nor the ability to issue new certificates. CI checks this layout.

3.2 User authentication

- Passwords are stored as salted scrypt hashes ($N = 2^{14}, r = 8, p = 1$) and compared in constant time. A login for an unknown user verifies against a dummy hash, so response time does not reveal which usernames exist.
- Failed logins are rate-limited with two counters stored in `gateway.db`: five failures in five minutes per (username, client address), and twenty per client address across all usernames. Either limit blocks further attempts for 60 seconds (`RESOURCE_EXHAUSTED`). Because the per-user counter includes the address, an attacker cannot lock the real user out from elsewhere, and the address-wide counter stops password spraying. A restart does not reset the counters.
- A successful login returns a JWT with issuer, subject, issue time, 15-minute expiry and a unique id (`jti`), signed with Ed25519 (`EdDSA`). Only the gateway holds the private key; a service that merely checks tokens needs only the public key and cannot mint one. Verification accepts `EdDSA` alone, so `alg: none` tokens and HS256 tokens "signed" with the public key are refused.
- Keys rotate without logging anyone out. Each token names its key in the `kid` header; the key directory holds `<kid>.key` (signs) and `<kid>.pub` (verifies only) files, the newest private key signs, and the gateway re-reads the directory when it changes. A rotation adds a key, waits one token lifetime, then retires the old key to its public half

(`scripts/rotate_jwt_key.py`). In Docker the keys live in a volume that only the gateway mounts.

- Logout revokes the token's `jti`. The revocation list is stored in `gateway.db`, so a revoked token stays invalid across restarts.

3.3 Authorization

`AuthorizationInterceptor` rejects every call without a valid, unexpired, unrevoked token (except `AuthenticateClient` and `RegisterBank`) with `UNAUTHENTICATED`. The service then enforces ownership: a user may only check balances of, and pay from, accounts linked to them (`PERMISSION_DENIED` otherwise), and may only see the status of payments they sent or received.

3.4 Input validation

The server validates every payment independently of the client:

- the amount must be between 1 paisa and Rs. 10,00,000 (banks additionally refuse non-positive amounts);
- the receiver account must exist (`NOT_FOUND`);
- sender and receiver accounts must differ;
- the transaction id must be 1–64 characters from `[A-Za-z0-9_-]`.

Each failure returns a specific gRPC status code (`INVALID_ARGUMENT`, `NOT_FOUND`, `PERMISSION_DENIED`) rather than a generic error.

4 Transaction Processing (2PC)

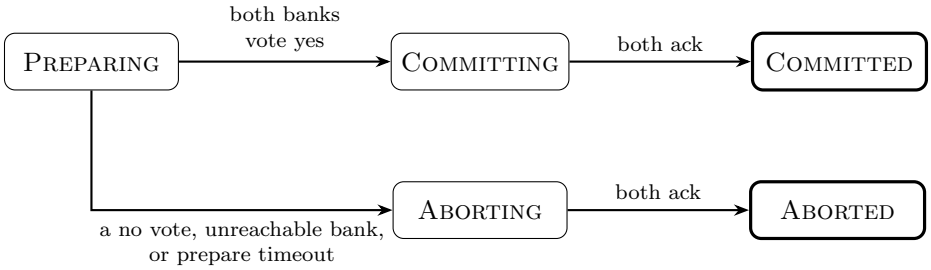


Figure 2: Payment states in the coordinator log. Every transition is a compare-and-set on the stored status.

4.1 Phase 0: log the payment

After validation the gateway inserts the payment into `gateway.db` with status PREPARING. The client-generated transaction id is the primary key.

4.2 Phase 1: prepare

The gateway asks, in parallel, the sender's bank to prepare a DEBIT and the receiver's bank to prepare a CREDIT. Each bank votes in one step on its single database writer, so the check and the reservation are serialised against every other write:

- **Debit:** vote yes only if balance − reserved ≥ amount, where *reserved* is the sum of the account's other prepared debits; the vote itself records a reservation of *amount*.

- **Credit:** vote yes if the account exists.

The vote is written to disk before the reply is sent. A repeated prepare for the same transaction returns the original vote.

4.3 Decision

If both banks vote yes, the gateway changes the status from PREPARING to COMMITTING; otherwise to ABORTING, storing the reason (for example "Insufficient funds"). The decision is durable *before* any bank is told about it.

4.4 Phase 2: commit or abort

The gateway sends the decision to both banks. Both operations are idempotent:

- **Commit** applies the debit or credit and marks the entry committed; repeating it is a no-op.
- **Abort** releases the reservation. An abort for a transaction the bank never saw writes a *tombstone*, so a delayed prepare that arrives afterwards is refused.

When both banks acknowledge, the status becomes COMMITTED or ABORTED. If a bank cannot be reached, the payment stays in COMMITTING/ABORTING and the recovery worker retries. A client whose payment is COMMITTING is told it succeeded: the decision is final and will be applied.

4.5 Timeouts and recovery

Every gateway-to-bank call has a 3-second deadline. The recovery worker runs every 2 seconds, and immediately when the gateway starts:

- COMMITTING/ABORTING: resend the decision to both banks until both acknowledge.
- PREPARING for longer than 30 seconds: the coordinator crashed or stalled before deciding. No commit decision exists, so the payment is moved to ABORTING (*presumed abort*) and the reservations are released.

Both the request thread and the recovery worker change state only by compare-and-set, so exactly one decision is ever recorded, even if they race.

5 Idempotent Payment Processing

5.1 Implementation

The client generates a random 128-bit transaction id for each payment and reuses it for every retry, including retries from the offline queue. The id is the primary key of the coordinator log:

- a new id starts a new payment;
- an id seen before *with the same user and parameters* returns the stored outcome (COMMITTED, ABORTED, or "in progress") without touching the banks;
- an id seen before with a different user or different parameters is rejected with `ALREADY_EXISTS`, so ids cannot be hijacked.

5.2 Correctness arguments

We assume the gateway and every bank eventually restart after a crash with their disks intact, and that the network eventually delivers messages between live processes.

Exactly-once (no double charge). Inserting a payment and checking for an existing id happen in one SQLite transaction, and the id is a primary key, so at most one payment record exists per id. The banks key their ledger entries on (transaction id, operation), so even a prepare or commit retried by the gateway affects each account at most once. Therefore a payment is applied at most once however often the client retries.

Atomicity. The decision is a single compare-and-set from PREPARING, so it is recorded exactly once and never changes. A bank applies a commit only for an entry it prepared, and only when the gateway sends COMMIT, which it does only after logging COMMITTING; a bank releases a reservation only on ABORT. Hence the two banks can never apply different outcomes. Because votes and decisions are durable, a crash of either side loses neither; and because phase 2 is idempotent and retried until acknowledged, both banks eventually apply the logged decision.

No overdraft or double spend. Each bank maintains the invariant balance $-$ reserved ≥ 0 for every account. Prepare checks it and adds the reservation in one serialised transaction, so two concurrent debits cannot both pass the check against the same funds. A committed debit subtracts an amount that was already reserved, so the balance never goes negative. A database `CHECK` constraint enforces this as well.

Conservation of money. Every committed payment applies a debit and a credit of the same amount, and by atomicity they are applied together or not at all. The total money across all banks is therefore constant. The chaos experiment (Section 9.2) checks exactly this.

Liveness and its limit. Under the assumptions above, every payment reaches COMMITTED or ABORTED. 2PC is, however, a *blocking* protocol: while the coordinator is down, a bank that voted yes must keep its reservation, because it cannot know the decision. With a single gateway the funds are safe but unavailable until it returns. Section 6 removes this limit by replicating the coordinator.

6 Replicated Coordinator

The blocking problem is a problem of the coordinator's *state*, not of the process: any process that can read the durable decision can finish the protocol. When `PG_ETCD_ENDPOINTS` is set, the coordinator log is therefore stored in a three-node etcd cluster instead of `gateway.db`. etcd replicates every write to a majority of its nodes with the Raft consensus protocol before acknowledging it, so the log survives the loss of any one node, and any number of gateways can share it. This is the approach of systems such as Spanner, whose 2PC coordinators are themselves replicated by consensus.

Design.

- **Same protocol, shared log.** Payments, the bank directory, revoked tokens and login failures are etcd keys. Every status change is an etcd transaction conditioned on the payment record's `mod_revision`: the same compare-and-set as with SQLite, now linearizable across gateways. No gateway owns a payment.
- **Recovery by anyone.** Every gateway runs the recovery worker over the shared log. Decided payments get a short grace period (one RPC timeout) so that the gateway running them is not duplicated by the others; after that, any gateway resends the decision. A payment abandoned in PREPARING is presumed aborted by whichever gateway sees it first.

- **Shared sessions.** Tokens are verified with the shared Ed25519 public keys, and revocations and lockouts are in etcd, so a session works at, and a logout or lockout applies to, every gateway. Revocations and failed-login records are attached to etcd leases and expire on their own.
- **Failover.** Clients and banks list every gateway; gRPC balances calls round-robin over the live ones, so a gateway crash fails only the calls in flight on it, and those are safe to retry with the same transaction id.
- **Isolation.** etcd requires mutual TLS from clients and between its Raft peers, with certificates from a CA of its own; only gateways hold one. A request whose reply is lost is retried; a payment insert carries a random attempt id so that a retried insert recognises its own earlier success.

Correctness. The arguments of the previous section carry over unchanged, because they rely only on the log being durable and on decisions being made by compare-and-set: etcd provides both, linearizably, for every gateway. Two gateways racing on one payment (one executing it, one recovering it) cannot both change its state from PREPARING; the loser re-reads the record and follows the recorded decision.

Availability. With three gateways and three etcd nodes, payments continue through the failure of any one gateway and any one etcd node. Payments involving a bank that is down still wait for that bank, since each bank is a single server.

Cost. Every state change is a consensus round, so a payment takes longer. On the laptop, one client completed about 26–30 payments/s (median 33 ms) against about 67/s (14 ms) with one gateway on SQLite; at 32 clients, about 182 against 256–270 payments/s.

7 Offline Payments & Retry Mechanism

7.1 Client-side queuing

- Each RPC is retried up to three times while the gateway reports `UNAVAILABLE` or the deadline expires.
- If the gateway is still unreachable, the payment, with its transaction id, is appended to `offline_payments.json` and survives client restarts.

7.2 Automatic resending

A background thread checks the channel's connectivity every 10 seconds (without sending fake requests) and, once the gateway is reachable and the user is logged in, resends queued payments. A payment leaves the queue once the gateway gives any final answer: completed, aborted for insufficient funds, or rejected as invalid. Because the transaction id is unchanged, a payment that had reached the gateway before the connection dropped is not applied a second time.

7.3 Failure handling

Failure	Behaviour
Gateway down	Client queues the payment; it is sent once the gateway returns.
Response lost after the gateway processed it	Resend returns the stored outcome; no second charge.
Bank down during prepare	Payment aborts; the other bank's reservation is released; the unreachable bank's abort is delivered when it returns.
Bank down during commit	Payment stays COMMITTING; recovery commits when the bank returns (its yes vote is on disk).
Gateway crash after the decision	On restart, recovery delivers the logged decision.
Gateway crash before the decision	After the prepare timeout, presumed abort releases the reservations.
Delayed prepare after an abort	Refused because of the abort tombstone.

8 Logging & Monitoring

- `LoggingInterceptor` records every RPC with its method, final status code, latency and peer address.
- `AuthorizationInterceptor` and `PeerIdentityInterceptor` log each rejected call and its reason.
- The coordinator logs each payment's progress (prepare votes, decision, phase-2 retries, recoveries). Banks log every vote, commit and abort.
- The authoritative history is the coordinator log in `gateway.db`, which `ViewTransactionHistory` and `GetPaymentStatus` query. Users see the sender's account number only on their own outgoing payments.

9 Evaluation

9.1 Automated tests

55 pytest tests, most of which start a gateway and three banks in-process on free ports:

- **Payments (7):** cross-bank transfer; insufficient funds; server-side rejection of zero, negative and oversized amounts; specific error codes; idempotent retries; a double-spend attempt with 40 concurrent Rs. 100 payments against a Rs. 1,500 balance (exactly 15 succeed); 120 random concurrent transfers that must conserve money.
- **Security (13):** missing, forged, expired and revoked tokens; revocation surviving a restart; login lock-out scoped to the client address and surviving a restart; password spraying; access to another user's account; bank impersonation; non-client certificates on user RPCs; non-gateway callers at a bank; the per-component key layout.
- **Recovery (7):** receiver commit failure; bank down during prepare; gateway crash after the decision; gateway crash before the decision; bank crash after voting yes; late prepare after abort; malformed operation.
- **Client (12):** exact rupee parsing, and an offline payment sent exactly once after the gateway returns.

- **Tokens (5):** EdDSA tokens naming their key; algorithm confusion, `alg: none` and wrong-issuer tokens refused; a key rotation that keeps existing sessions valid.
- **Storage (5):** concurrent writes share commits; a failing write rolls back only itself; data is on disk when `write()` returns.
- **Replicated coordinator (6),** against a real three-node etcd cluster: sessions, payments, idempotent retries, logout and lock-outs across gateways; one gateway completing, or presuming aborted, payments left by another that crashed; payments continuing with an etcd node down; racing compare-and-sets with exactly one winner.

9.2 Chaos experiment

`scripts/chaos_demo.py` runs the gateway and the three banks as separate operating-system processes. Eight client threads send random payments while, every 1–3 seconds, a random component is hard-killed (`SIGKILL`/`TerminateProcess`, so no clean-up code runs) and restarted. After the load stops and recovery settles, an audit compares the coordinator log with every bank ledger. Result of a 40-second run:

Measure	Value
Components killed (gateway / HDFC / ICICI / SBI)	15 (4 / 5 / 2 / 4)
Payments logged	2,747
committed / aborted	2,264 / 483
Client calls that failed with `UNAVAILABLE`	32
In-doubt payments settled by the recovery worker	158 (117 committed, 41 aborted)
of which presumed aborted after a gateway crash	10
Payments left in doubt, or reservations left held	0
Payments where the two banks disagree	0
Money before / after	Rs. 23,000.00 / Rs. 23,000.00

The aborts combine insufficient-funds outcomes (the workload draws random amounts against small balances) with payments that a kill interrupted before a commit decision.

The same experiment with `--replicated` runs three etcd nodes, three gateways and the three banks, and kills any of them, one at a time. A 60-second run killed 22 components (5 gateways, 9 etcd nodes, 8 banks) during 4,164 payments (3,433 committed, 731 aborted). The audit again found every payment atomic, nothing in doubt and the money unchanged. Recovery workers settled 46 in-doubt payments, often on a gateway other than the one that had started them, and presumed 7 abandoned payments aborted. Only 0.3% of client calls failed with `UNAVAILABLE`, against 0.6% in a 60-second single-gateway run: with several gateways, only the calls in flight on the killed one fail.

9.3 Throughput and latency

`scripts/benchmark.py` runs the same multi-process deployment (mTLS on every hop) and measures completed payments per second at several client concurrencies (8 seconds per level, Windows laptop). Each payment performs four gateway-to-bank calls and seven durable writes: three at the gateway and two at each bank.

Each process commits its writes through a single writer thread with *group commit*: the writer takes every write that is waiting, runs each in its own savepoint, and commits the batch with one fsync. No caller is answered before that commit, so durability is unchanged, but concurrent payments share flushes.

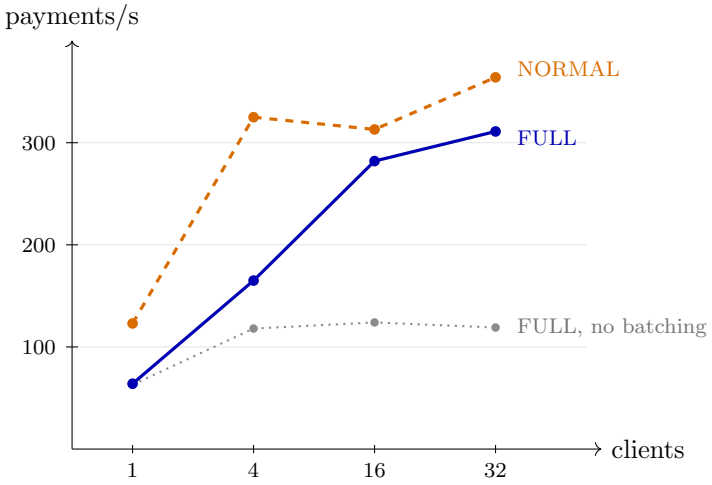


Figure 3: Throughput with group commit under SQLite `synchronous=FULL` (fsync on every commit, the default) and `NORMAL`, against the previous one-fsync-per-write design under `FULL`.

	`FULL` (default)		`NORMAL`	
Clients	TPS	p50 / p99 (ms)	TPS	p50 / p99 (ms)
1	64	14 / 25	123	8 / 16
4	165	24 / 37	325	12 / 23
16	282	52 / 105	313	48 / 100
32	311	98 / 350	364	87 / 134

Discussion. With one fsync per write, throughput under full durability levelled off at about 120–140 payments/s regardless of the number of clients: each process flushed its writes one at a time, so extra clients only queued. Group commit removes that ceiling. In interleaved runs against the previous code, throughput at 16 clients rose from 63–128 to 282–297 payments/s, and at 32 clients from 133–148 to 311–352. It is now close to the `NORMAL` figures, which trade the last commits on an operating-system crash for speed, so that trade-off is no longer needed for throughput. A single client gains nothing, since there is nothing to batch. Below about four clients the results vary by up to 2–3× between runs on this laptop because flush latency is noisy, so comparisons use interleaved A/B rounds (`benchmark.py --ab PG_GROUP_COMMIT 0 1`).

Parallel prepare. The gateway asks both banks to prepare at the same time. An A/B comparison alternated three runs each of sequential and parallel prepare under `NORMAL`. Parallel prepare cut single-client median latency from 10.2 ms to 8.1 ms and raised single-client throughput from 92 to 110 payments/s. At 4–16 clients the two were within run-to-run noise: there, throughput is limited by serialised database writes rather than by the extra round trip. Under `FULL` the fsync noise exceeded the difference.

10 Assumptions

1. Users are predefined in `users.json` (with hashed passwords); there is no self-registration. New users are added with `scripts/hash_password.py`.
2. A user has at most one account per bank, so a bank name identifies the account unambiguously.
3. Every component has a certificate issued by the deployment's CA before it starts.

4. Banks and clients know the gateway's address (`PG_GATEWAY_ADDRESS`); banks tell the gateway their own address when they register.
5. Crashed processes eventually restart with their disks intact.

11 Deployment

`docker compose up` runs a one-shot job that creates the CA and writes each component's credentials into its own volume, then starts the gateway and the three banks, each with its own credentials and data volumes. The client runs with `docker compose run client`. GitHub Actions installs etcd (`scripts/get_etcd.py` checks the release's SHA-256) and runs the test suite, the chaos experiment in both layouts, a check that the generated protobuf code is up to date, and a Docker Compose smoke test that also verifies each container holds only its own key.

12 Limitations & Future Enhancements

- **Participant availability.** The coordinator can be replicated, but each bank is a single server, so a payment involving a bank that is down waits for it. A real bank would replicate its ledger as well.
- **Clocks.** Gateways compare wall-clock times to decide that an undecided payment has timed out. Large skew could cause an early, and still atomic, abort.
- **Deployment.** Docker Compose runs a single gateway on SQLite; the replicated layout runs as local processes.
- **Storage.** Group commit shares fsyncs, but each process still applies its writes one at a time on one SQLite connection. PostgreSQL with row-level locking would let independent accounts proceed in parallel.
- **Identity.** Users come from a JSON file, and token signing keys are files on disk. A production system would use a user database and keep signing keys in a KMS or HSM.
- **Operations.** Metrics such as Prometheus counters for payment outcomes and recovery backlog, and an admin view of in-doubt payments.